SIMATS ENGINEERING
CO5-ASSESSMENT TOOL -15
Saveetha Institute of Medical and Technical Sciences
Chennai- 602105

Student Name: TALARI VISHNUVARDHN **Reg. No.: 192572091**

Course Code: **CSA1511** **Slot:** B

Course Name: **CLOUD COMPUTING AND BIG DATA ANALYTICS**

Course Faculty: Dr. Hemavathi R

1. Hadoop Job Fails Because Input Splits Are Uneven and Reducers Remain Idle

Answer:

A Hadoop job may repeatedly fail or perform poorly when the **input splits are uneven**. Some Mapper tasks receive very large amounts of data while others receive very small amounts. This creates an unbalanced workload.

Likely Design Error

The main design error is **poor input partitioning and improper task distribution**.

Uneven input splits cause:

- Some Mapper tasks to take a long time.
- Some Mapper tasks to finish early.
- Cluster resources to remain underutilized.
- Reducers to remain idle while waiting for Mapper output.
- Overall job execution time to increase.

Corrective Actions

1. Create Balanced Input Splits

The input data should be divided into approximately equal-sized splits so that Mapper tasks receive a balanced workload.

2. Improve Data Partitioning

A proper partitioning strategy should be used to distribute intermediate keys evenly among Reducers.

3. Configure an Appropriate Number of Reducers

The number of Reducer tasks should be selected based on the amount of data and available cluster resources.

4. Use a Custom Partitioner

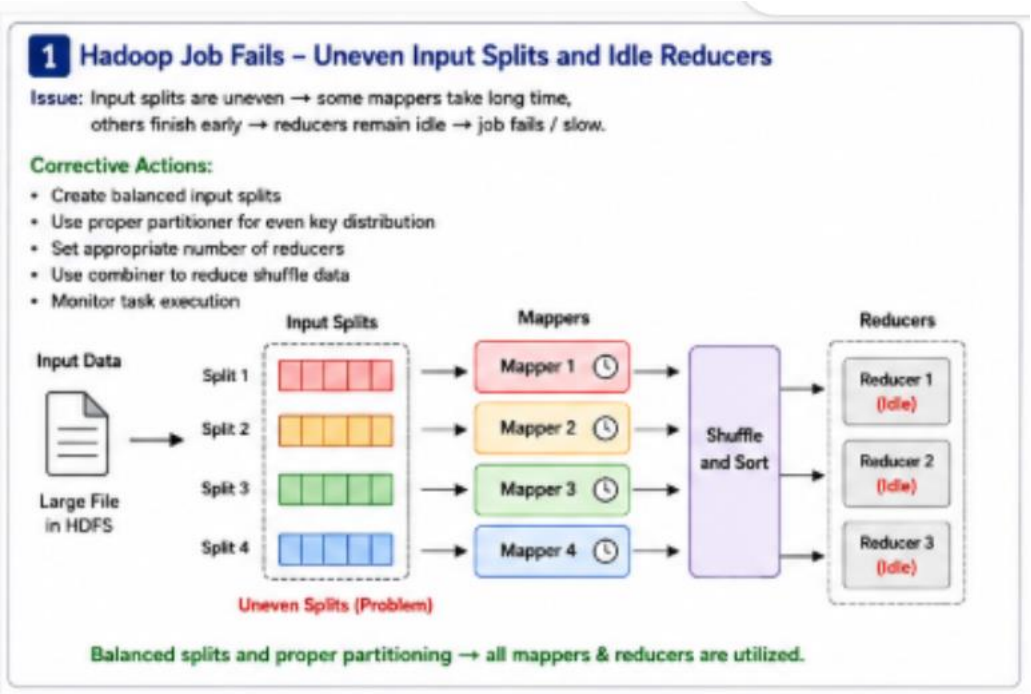
If some keys contain significantly more data than others, a custom partitioner can distribute the workload more evenly.

5. Monitor Job Performance

Hadoop monitoring tools can be used to identify slow Mapper or Reducer tasks.

Conclusion

The likely problem is **uneven input splitting and poor workload distribution**. By creating balanced splits, improving partitioning, and properly configuring Reducers, the Hadoop job can use cluster resources efficiently and complete successfully.



2. HDFS Shows Data Unavailability After a Single Node Failure

Answer:

HDFS is designed to provide fault tolerance. Therefore, frequent data unavailability after the failure of only one node indicates a problem in the **replication strategy or NameNode design**.

Probable Design Errors

1. Insufficient Replication Factor

The replication factor may be set too low. If only one copy of a data block exists, failure of the DataNode storing that block will make the data unavailable.

2. Improper Replica Placement

Multiple replicas may be stored on the same node or in locations that are affected by the same failure.

3. NameNode Single Point of Failure

If only one NameNode is used and it fails, the HDFS cluster may become unavailable.

Corrective Actions

1. Use an Appropriate Replication Factor

Important data blocks should have multiple replicas stored on different DataNodes.

2. Use Rack Awareness

Replicas should be distributed across different racks to protect against rack-level failures.

3. Implement NameNode High Availability

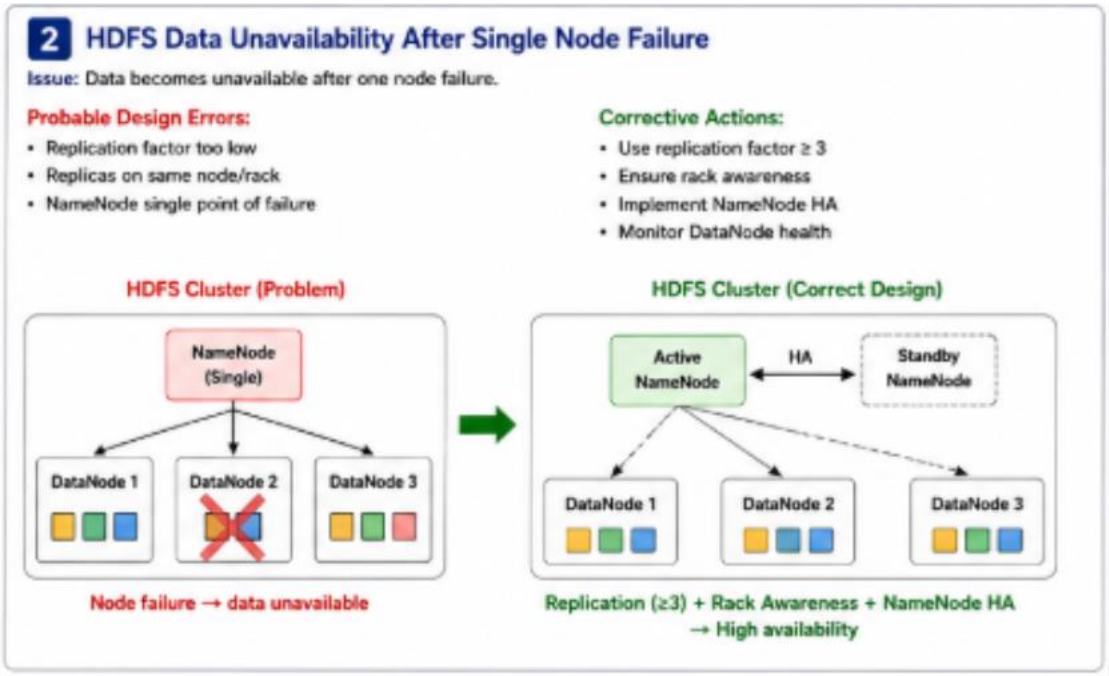
Use an **Active NameNode and Standby NameNode**. If the Active NameNode fails, the Standby NameNode can take over.

4. Monitor DataNode Health

The cluster should continuously monitor DataNodes and automatically create new replicas when failures occur.

Conclusion

The main cause is likely an **incorrect replication strategy or lack of NameNode High Availability**. Proper replication, rack awareness, and NameNode HA can significantly improve data availability.



3. ETL and Apache NiFi Workflow Creates Duplicate Records

Answer:

Duplicate records in an analytics store indicate a problem in the **data ingestion workflow**. The duplication may occur when the same data is processed or transferred multiple times.

Likely Causes

1. Repeated Data Ingestion

The ETL process may read the same source data more than once.

2. NiFi Flow Reprocessing

A NiFi processor may send the same FlowFile again because of retries or incorrect flow configuration.

3. Missing Unique Record Identification

The system may not check whether a record already exists before storing it.

4. Failure During Data Transfer

If a failure occurs after data is partially stored, the workflow may retry and insert the same records again.

Recommended Fixes

1. Use Unique IDs

Each record should have a unique identifier to detect duplicates.

2. Implement Deduplication

A deduplication process should check incoming records before storing them in the analytics system.

3. Use Proper NiFi Flow Management

Configure Apache NiFi processors carefully to avoid unnecessary repeated processing.

4. Maintain Processing Status

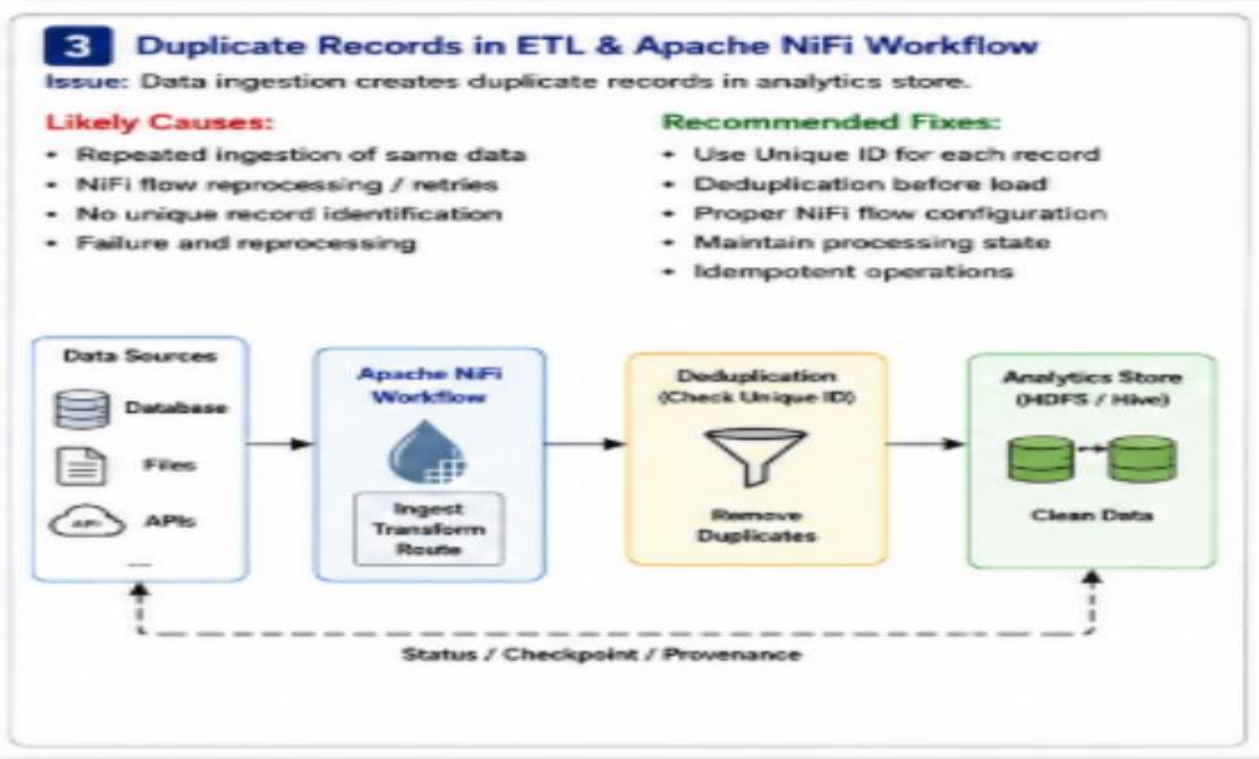
Store information about successfully processed records so that they are not processed again.

5. Use Idempotent Operations

The ingestion process should be designed so that processing the same record multiple times does not create multiple copies.

Conclusion

Duplicate records are mainly caused by **repeated ingestion, retry operations, or missing duplicate checks**. Unique IDs, deduplication, and proper NiFi flow configuration can solve the problem.



4. YARN Cluster Suffers from Resource Starvation

Answer:

Resource starvation occurs when multiple users submit jobs at the same time and some applications do not receive enough CPU or memory resources.

Likely Scheduling Error

The main problem is an **improper resource allocation and scheduling policy**.

Possible causes include:

- No proper resource limits.
- One user or application consuming most resources.
- Lack of separate queues.
- No priority management.
- Poor configuration of CPU and memory allocation.

Better Resource Management Approach

1. Use Capacity Scheduler

The Capacity Scheduler can divide cluster resources among different users or applications.

2. Create Separate Queues

Different queues can be created for different job types or users.

For example:

- High Priority Queue
- Normal Priority Queue
- Low Priority Queue

3. Define Resource Limits

Maximum CPU and memory limits should be assigned to each queue or user.

4. Ensure Fair Resource Allocation

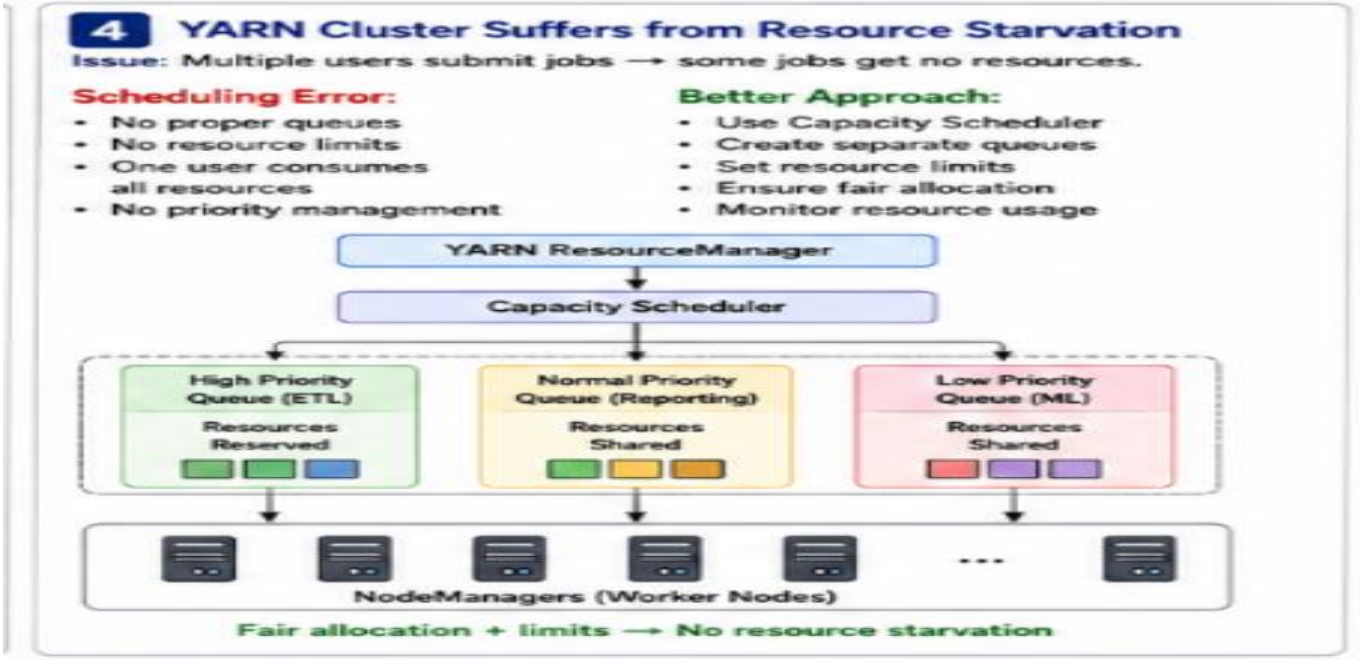
Resources should be distributed fairly so that one user cannot consume the entire cluster.

5. Monitor Cluster Resources

The ResourceManager should monitor resource usage and allocate resources based on availability.

Conclusion

The resource starvation problem is caused by **poor scheduling and uncontrolled resource allocation**. Using the Capacity Scheduler, separate queues, resource limits, and fair allocation can improve YARN cluster performance.



5. Backup Strategy Restores Outdated Data After Recovery

Answer:

If a distributed storage system restores outdated data after recovery, the problem is likely related to an **incorrect backup or replication design**.

Probable Design Flaws

1. Backups Are Not Updated Regularly

The system may restore an old backup because recent backups were not created.

2. Replication Is Confused with Backup

Data replication provides availability but does not always provide historical backup versions.

3. No Version Control

The system may not maintain different versions of data.

4. Backup Synchronization Failure

Recent data changes may not have been synchronized with the backup storage.

Recommended Fixes

1. Schedule Regular Backups

Automatic backups should be created at regular intervals.

2. Maintain Backup Versions

Multiple backup versions should be stored so that the latest correct version can be selected.

3. Verify Backup Consistency

Backup data should be checked regularly to ensure that it is complete and current.

4. Separate Replication and Backup

Replication should be used for availability, while a proper backup system should be used for recovery.

5. Use Recovery Testing

Regular recovery tests should be performed to verify that the latest data can be restored successfully.

Conclusion

The main design flaw is likely **outdated backup synchronization or confusion between replication and backup**. Regular automated backups, version management, consistency checking, and recovery testing can ensure that the latest data is restored.